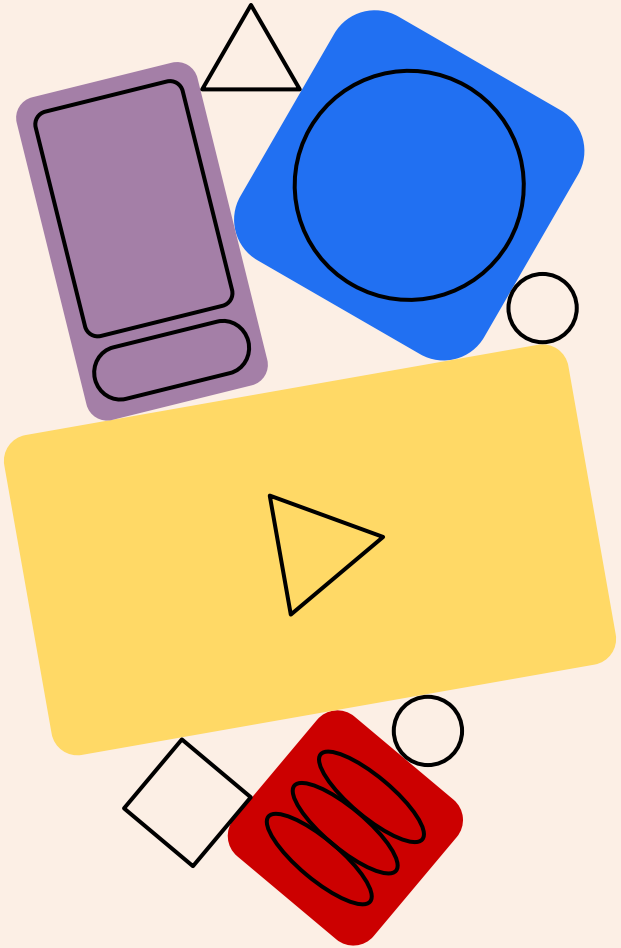
Green LIME

Improving AI Explainability through Design of Experiments

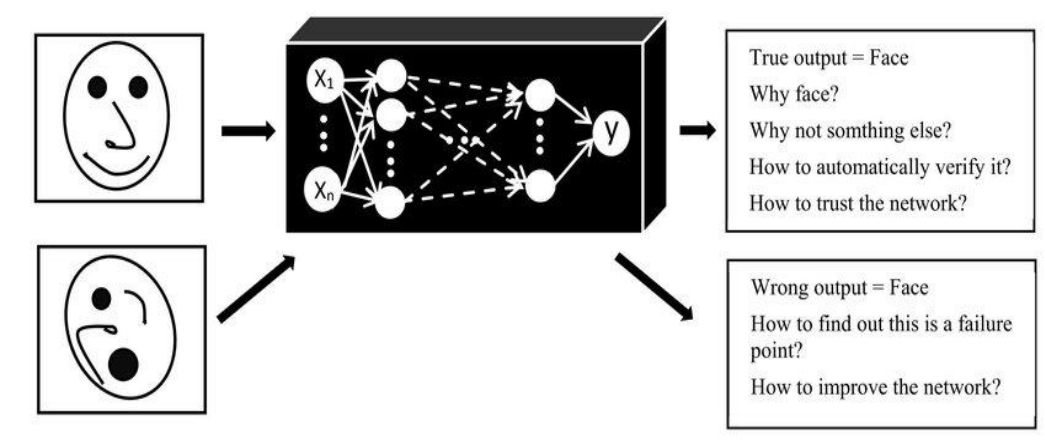


Why Explainability is Needed in AI?

Modern AI models are complex black boxes.

We cannot understand:

- Why a prediction was made
- Which features influenced it



Why this matters?

Trust requires **explainability.**

For example, in following fields we need explanation of certain prediction:

- Healthcare
- Finance
- Legal systems

Primary Model vs **Secondary** Model

1. Primary Model (The "Black-Box")

Definition: A complex machine learning model whose reasoning exceeds human understanding and requires explanation.

Mathematical Function:

$$y = f(x), \quad x \in \mathbb{R}^m,\ y \in \mathbb{R}$$

Scope & Nature: Operates on the **global model domain**; the functional form and prediction surface of fff are typically **unknown**.

2. Secondary Model (The Interpretable Surrogate)

Definition: A simple, interpretable model used to approximate the primary model locally.

Mathematical Function:

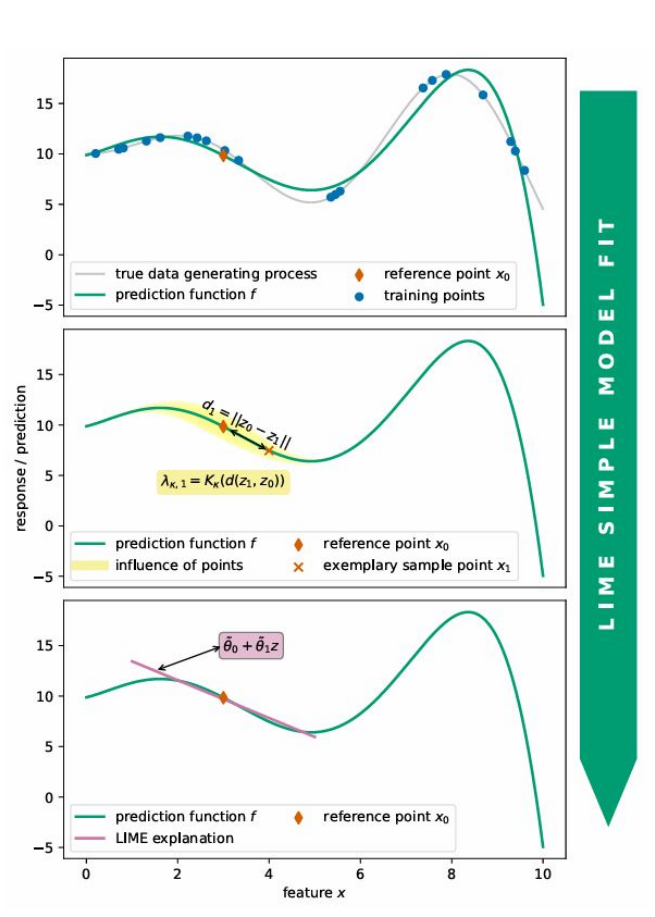
$$\boxed{y = h(z)^\top \theta + \epsilon}$$

$$h(z)^\top = (1, z^\top), \quad \theta \in \mathbb{R}^{m+1}, \quad \epsilon \text{ is the remainder}$$

Optimization (Weighted Least Squares):

$$\tilde{\theta} = \arg\min_{\theta} \sum_{i=1}^{N} \lambda_{\kappa,i} \left(y_i - h(z_i)^\top \theta\right)^2$$

Scope & Nature: Model-agnostic, operates **locally** around x_0; common types include **linear regression and decision trees**.



Overview of LIME

- **Model-Agnostic Framework:** LIME explains individual predictions by fitting an interpretable **secondary model** (e.g., linear regression) to a complex primary model's output at the instance level.

- **Kernel-Based Locality:** It employs a **kernel function** (defaulting to RBF) and distance metrics to weight samples, ensuring the surrogate model maintains high local fidelity to the reference instance.

- **Stability & Efficiency Limitations:** Standard random sampling often leads to result instability (variability across runs) and high energy consumption due to evaluating numerous irrelevant points with low locality weights

Detailed Maths behind **LIME**

1. Primary Model Definition

$$y = f(x), \quad x \in \mathbb{R}^m,\ y \in \mathbb{R}$$

Meaning: f is the complex model, x is input vector, y is predicted output.

2. Data Standardization

$$\omega_j(x) = \frac{x_j - \bar{x}_j}{s_j}$$

Standardized instance:

$$z_0 = \omega(x_0)$$

Purpose: ensures all features are on same scale.

3. Neighborhood Sampling

$$\check{x}_i \sim N(x_0, \Sigma)$$

where Σ is diagonal covariance matrix with variances s_j^2.

4. Local Interpretable Model

$$y = h(z)^T\theta + \epsilon$$

$$h(z)^T = (1, z^T), \quad \theta \in \mathbb{R}^{m+1}$$

This model approximates f locally.

5. Kernel and Locality Weighting

$$d(z_1, z_2) = \sqrt{(z_1 - z_2)^T(z_1 - z_2)}$$

$$K_\kappa(d) = \exp\left(-\frac{d^2}{2\kappa^2}\right)$$

$$\lambda_{\kappa,i} = K_\kappa(d(z_i, z_0))$$

Nearby points receive higher weight.

6. Optimization

$$\tilde{\theta} = \arg\min_\theta \sum_{i=1}^{N} \lambda_{\kappa,i}(y_i - h(z_i)^T\theta)^2$$

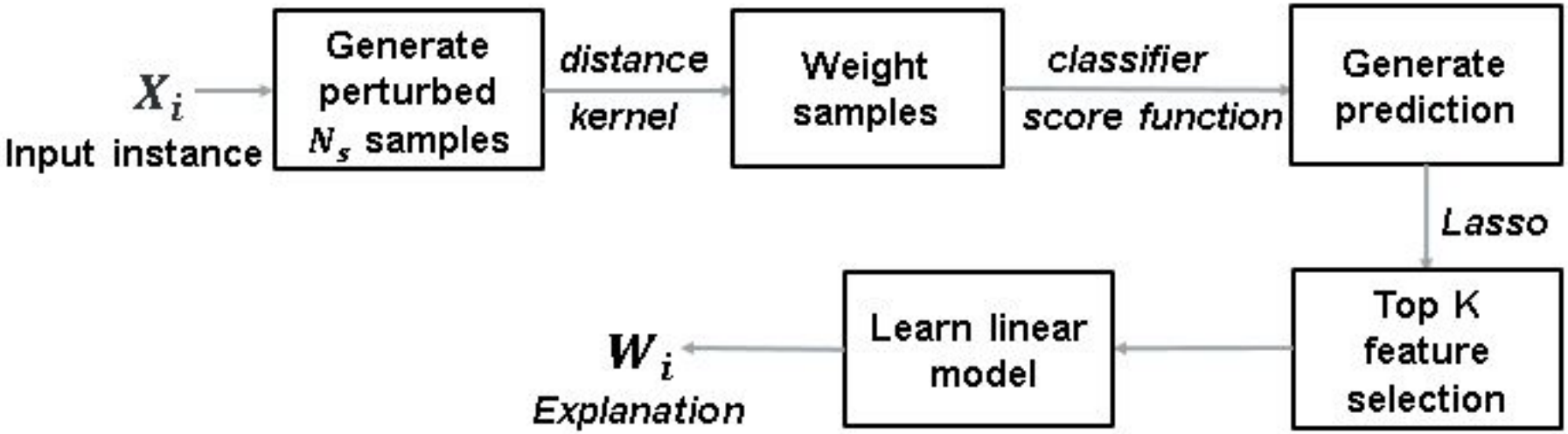
where $y_i = f(\check{x}_i)$.

7. Explanation Output

$$\begin{bmatrix} z_{01}\tilde{\theta}_1 \\ z_{02}\tilde{\theta}_2 \\ \vdots \\ z_{0m}\tilde{\theta}_m \end{bmatrix}$$

Each term represents contribution of a feature.

LIME Mind-Map



Weighted Least Squares for Explanation

Locality Weighting (λκ,i): Each sampled point is assigned a weight based on its distance from x_0. Nearby points get higher weights, distant points get lower weights, ensuring a local explanation.

Optimization Goal: Find θ by minimizing the weighted squared error between primary model predictions and secondary model outputs:

$$\tilde{\theta} = \text{argmin}_\theta \sum_{i=1}^{N} \lambda_{\kappa,i}(y_i - h(z_i)^\top \theta)^2$$

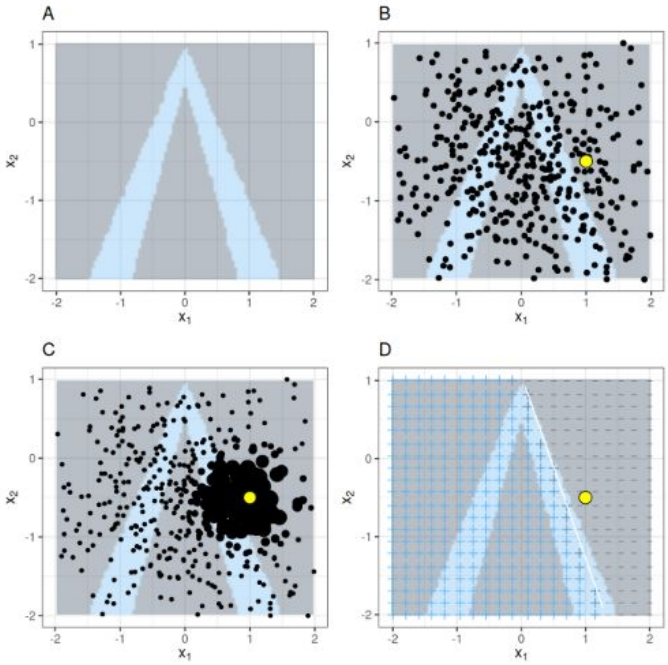


Figure 14.1: LIME algorithm for tabular data. A) Prediction surface given features x_1 and x_2. Predicted classes: 1 (dark) or 0 (light). B) Instance of interest (big dot) and sampled data (small dots). C) Assign weights based on distance to instance. D) Signs (+/-) show the classifications of the locally learned model from the weighted samples. The white line marks the decision boundary (P(c=1) = 0.5).

Interpretation of LIME Coefficients

- LIME coefficients represent the contribution of each feature to the prediction of the instance x_0.
- Each coefficient $\tilde{\theta}_j$ shows the local effect of a feature. A positive value increases the prediction, while a negative value decreases it.
- Larger absolute values $|\tilde{\theta}_j|$ indicate more influential features, allowing feature importance ranking.
- These coefficients are valid only near x_0 and do not describe the global behavior of the model.
- **Mathematical Representation:**

$$(z_{01}\tilde{\theta}_1,\ z_{02}\tilde{\theta}_2,\ \ldots,\ z_{0m}\tilde{\theta}_m)^T$$

This vector represents the feature-wise contribution to the prediction.

Random Sampling from Multivariate Normal Distribution

Sampling Distribution: Synthetic points are generated from a multivariate normal distribution:

$$\check{x}_i \sim \mathcal{N}(x_0, \Sigma)$$

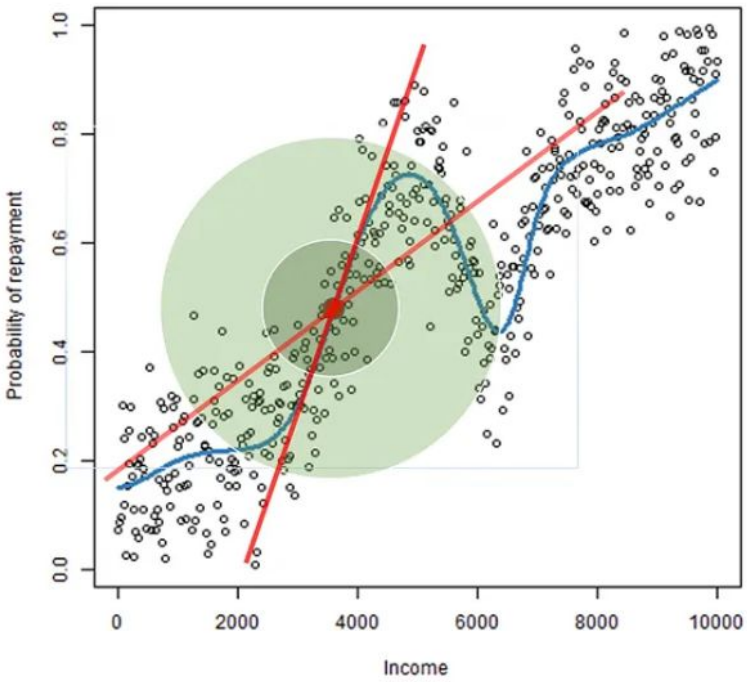
Center (x_0: The distribution is centered at the instance of interest x_0, ensuring samples lie in its local neighborhood.

Covariance Matrix (Σ): Σ is a diagonal matrix with entries (sj)^2, the variances of training features values, which control the spread of sampling.

Feature Independence: Since Σ is diagonal, feature values are sampled independently. Sampled points are passed through the primary model to create a local dataset for training the surrogate model.

$$y_i = f(\check{x}_i)$$

Drawbacks: Random sampling can cause instability and inefficiency, as distant samples may receive very low weights and not contribute to the explanation.



Why covariance matrix is diagonal in LIME?

What if it is not?

Variance of Training Features in LIME: Variance refers to how much the feature values vary in the **training data**. LIME uses this **variance** to **sample new values** around the **instance** x_0, keeping samples realistic and close to the original data distribution.

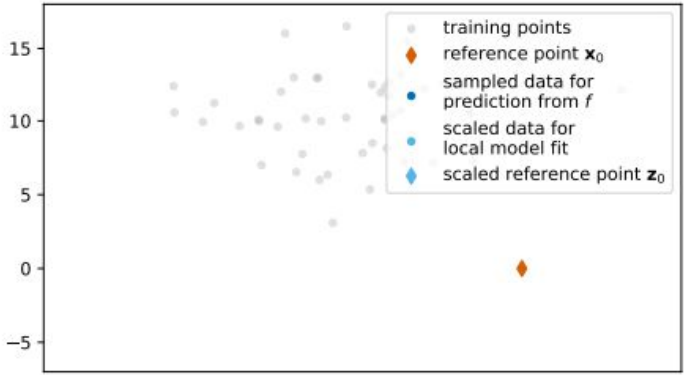
How much variance should be used: Variance should be moderate. Too large can make samples go far from instance, losing local behavior and too small can make the samples too close, **failing** to capture **feature influence properly.**

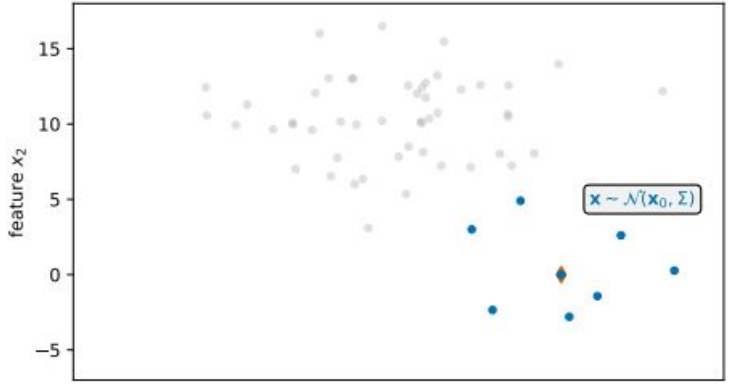
Why Covariance Matrix is Diagonal in LIME: A diagonal covariance matrix assumes features are independent, allowing LIME to vary each feature separately and measure its individual effect clearly. Assuming features are independent means LIME changes **one feature at a time** without considering its relationship with other features during sampling.

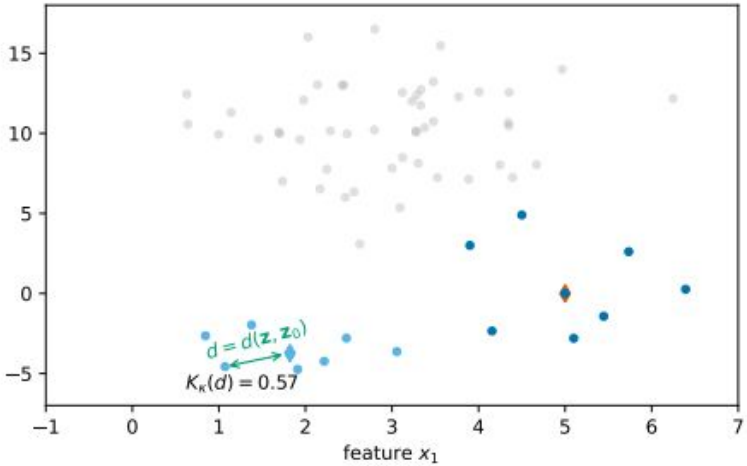
If covariance is not diagonal: Features become correlated during sampling, making it difficult to isolate and interpret the individual contribution of each feature.

Note: In real dataset, features may be correlated, so independent sampling can sometimes create unrealistic samples.

LIME: Pictorial Explanation







How LIME can be improved?

- **Sampling strategy**
  - Replace purely random sampling with more informative and structured sampling.
  - Improve coverage of the local neighborhood to produce more reliable surrogate models.

- **Distance measurement and locality definition**
  - Use learned embeddings (e.g., via autoencoders) instead of raw input space.
  - Provides a more meaningful notion of similarity between instances.

- **Kernel width selection**
  - Optimize kernel width rather than relying on fixed heuristics.
  - Ensures a better balance between locality and explanation stability.

- **Adaptive sample size determination**
  - Dynamically adjust the number of samples based on explanation stability.
  - Reduces unnecessary computation while maintaining explanation reliability.

Problem: Computational Cost and Inefficiency of LIME

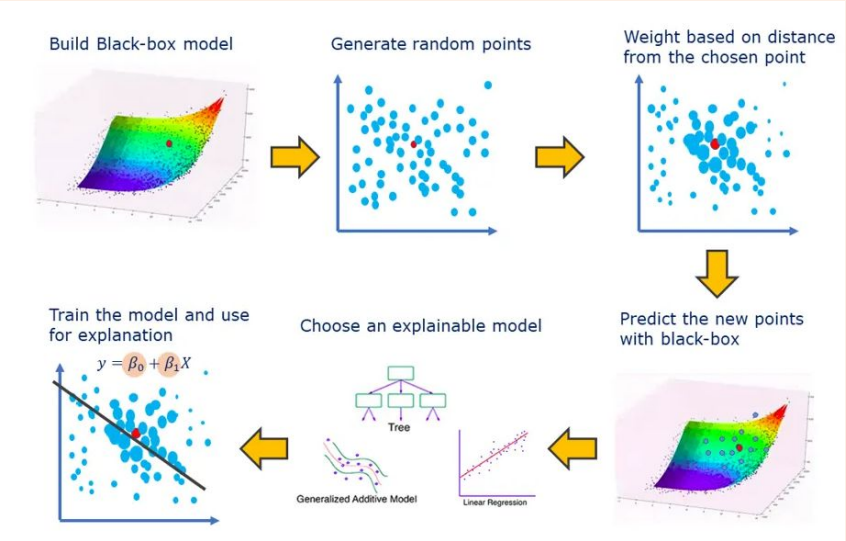
LIME explains predictions using local linear models.

But LIME requires:

- Many sampled points
- Many model evaluations

This leads to:

- High **computational cost**
- High **energy consumption**
- Slow explanations



Problem: Instability of LIME

Instability in LIME refers to the fact that the method leads to different results for multiple runs of the algorithm.

Why does this happen?

- sampling of neighborhood data is completely random, hence the training data for the interpretable secondary model is different at each run.

For example:

- A variable could for example positively affect the prediction in one run and have a negative impact in another run.

Consequence:

- Less trust in the model even though the main proposed purposes of LIME is to increase trust in a model

Problem: Excessive Model Evaluations

LIME requires evaluating many sampled instances, including irrelevant ones, which increases computational cost, runtime, and energy consumption.

Why does this happen?

- Many sampled instances are far from the instance of interest
- These instances have near-zero locality weight and they provide little or no useful information for the local surrogate model
- Despite this, the prediction function f must still be evaluated

For example:

- An explanation may require thousands of model evaluations (e.g., 5000 samples)
- Each evaluation adds computational cost, especially if the prediction function is expensive

Consequence:

- Increased computational runtime and energy consumption
- Higher deployment cost due to repeated inference
- Reduced efficiency, especially in real-time explanation scenarios

Key Idea: Using Optimal Design to Improve LIME

Goal:

- Reduce number of samples
- maintaining explanation quality

Solution:

- Use Optimal Design of Experiments

Key principle:

- Maximize information
- Minimize variance

Key Words: *samples, explanation quality, optimal design, information, variance*

Introduction to Optimal Design of Experiments

Optimal Design of Experiments (ODE) is a statistical approach focused on **maximizing information from limited observations**. It Improves **data collection** efficiency under constraints like computational resources.

It is Used in LIME to optimize selection of sample points for approximating the primary model with an interpretable secondary model. **Strategically selected samples** reduce primary model evaluations, **lowering computational cost** and energy consumption.

It **enhances the Stability** as it provides fixed support points, reducing variability and improving stability **compared to random sampling in LIME.**

Key-word: D-Optimal Design

The cube represents the design space of features ($X1$, $X2$, $X3$). The orange boundary points represent optimally selected support points using Optimal Design (D-optimal design).

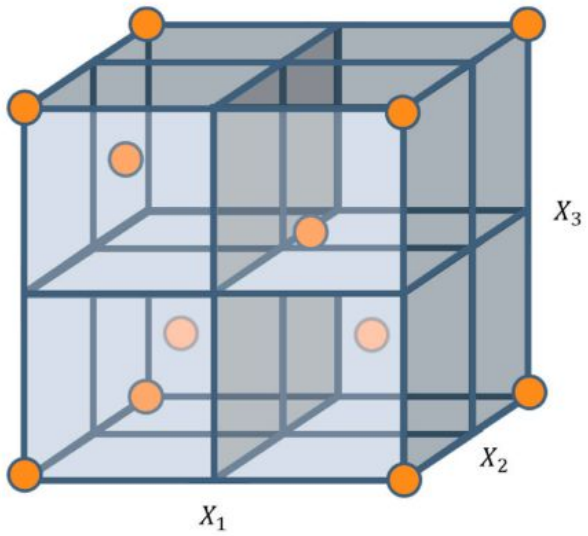


Fig. 4. Design space with selected input data combinations that serve as features for a metamodel.

Model Setup and Notation

Goal: Local approximation of the black-box model around an instance of interest

- x_0: instance of interest in original input space
- $z_0 = \omega(x_0)$: interpretable representation of x_0
- $z_i \in \mathbb{R}^m$: sampled interpretable instances
- $y_i = \eta(z_i) + \varepsilon_i$: model prediction with noise
  - $\eta(\cdot)$: true prediction function and ε_i: random error term
- $u_i = z_i - z_0$: shifted samples

Local linear surrogate model:

$$y_i = h(u_i)^\top \gamma + \varepsilon_i$$

- $h(u_i)^\top = (1, u_i^\top)$: feature vector (intercept + features)
- $\gamma \in \mathbb{R}^{m+1}$: local model parameters

Parameter Estimation

Goal: Weighted least squares estimation of local parameters

- $d(z_i, z_0)$: distance to reference point
- $\lambda_{\kappa,i} = K_\kappa(d(z_i, z_0))$: locality weight
  - $K_\kappa(\cdot)$: kernel function
  - κ: kernel width
- N: number of sampled instances

Parameter estimation:

$$\tilde{\gamma} = \arg\min_{\gamma} \sum_{i=1}^{N} \lambda_{\kappa,i} \left(y_i - h(u_i)^\top \gamma \right)^2$$

- $\tilde{\gamma}$: estimated surrogate parameters

Design and Optimality Criterion

Goal: Optimal sampling design for efficient explanation

Design representation:

$$\xi = \{z_q, p_q\}_{q=1}^{N^*}$$

- z_q: design points, p_q: design weights, and N^*: # of candidate points

Information matrices:

$$M_{11}(\xi) = \sum_{q=1}^{N^*} p_q \lambda_{\kappa,q} h(u_q) h(u_q)^\top, \quad M_f(\xi) = \sum_{q=1}^{N^*} p_q \lambda_{\kappa,q}^2 h(u_q) h(u_q)^\top$$

Mean squared error matrix:

$$R(\xi) = M_{11}^{-1}(\xi) M_f(\xi) M_{11}^{-1}(\xi)$$

Optimal design:

$$\xi^* = \arg\min_{\xi} \log |R(\xi)|$$

Approximation Goal

- The prediction function f is deterministic, meaning the same input always produces the same output
- Example: suppose a trained neural network predicts

$$f(x_0) = 0.87$$

  If we evaluate $f(x_0)$ again, it will still return 0.87, because the network weights are fixed
- Any variation in predictions comes only from changing the input, not from randomness in the function itself
- Therefore, approximation error arises from model misspecification rather than stochastic noise
- The goal is to approximate the average behavior of the prediction function in the neighborhood of the reference point
- This creates a trade-off between accurately representing behavior at the reference point and capturing behavior in its neighborhood

Importance of the Center Point

- The approximation is performed using shifted variables:

$$u_i = z_i - z_0$$

- The reference point corresponds to the center point:

$$u = 0$$

- The center point is always included in the sample to improve robustness and stabilize the surrogate model
- A fixed design weight is assigned to the center point:

$$\delta = \frac{1}{N+1}$$

- Including the center point also regularizes the optimal design and prevents sampling only distant points

Structure of the D-opt Design

- In one dimension, the optimal design consists of three points symmetrically placed around the center
- The optimal design is given by:

$$\xi^* = \begin{Bmatrix} -u, & 0, & u \\ \frac{1-\delta}{2}, & \delta, & \frac{1-\delta}{2} \end{Bmatrix}$$

- The center point receives weight δ, and the remaining weight is equally assigned to the symmetric points
- The optimal distance u^* is obtained by optimizing the D-optimality criterion
- The optimal distance scales with kernel width:

$$u_\kappa^* = \kappa u^*$$

Jittering for Practical Sample

- The optimal design may require multiple samples at the same design points
- Since the prediction function is deterministic, repeated evaluations at identical points do not provide additional information
- Jittering generates additional samples by adding small Gaussian noise to optimal design points
- The center point is included directly, and remaining samples are generated around optimal support points
- This allows flexible sample sizes while preserving the optimal design structure
- Jittering improves efficiency, stability, and robustness of the LIME sampling process

Jittering for Practical Sample

- The optimal design may require multiple samples at the same design points
- Since the prediction function is deterministic, repeated evaluations at identical points do not provide additional information
- Jittering generates additional samples by adding small Gaussian noise to optimal design points
- The center point is included directly, and remaining samples are generated around optimal support points
- This allows flexible sample sizes while preserving the optimal design structure
- Jittering improves efficiency, stability, and robustness of the LIME sampling process

Green-LIME Example

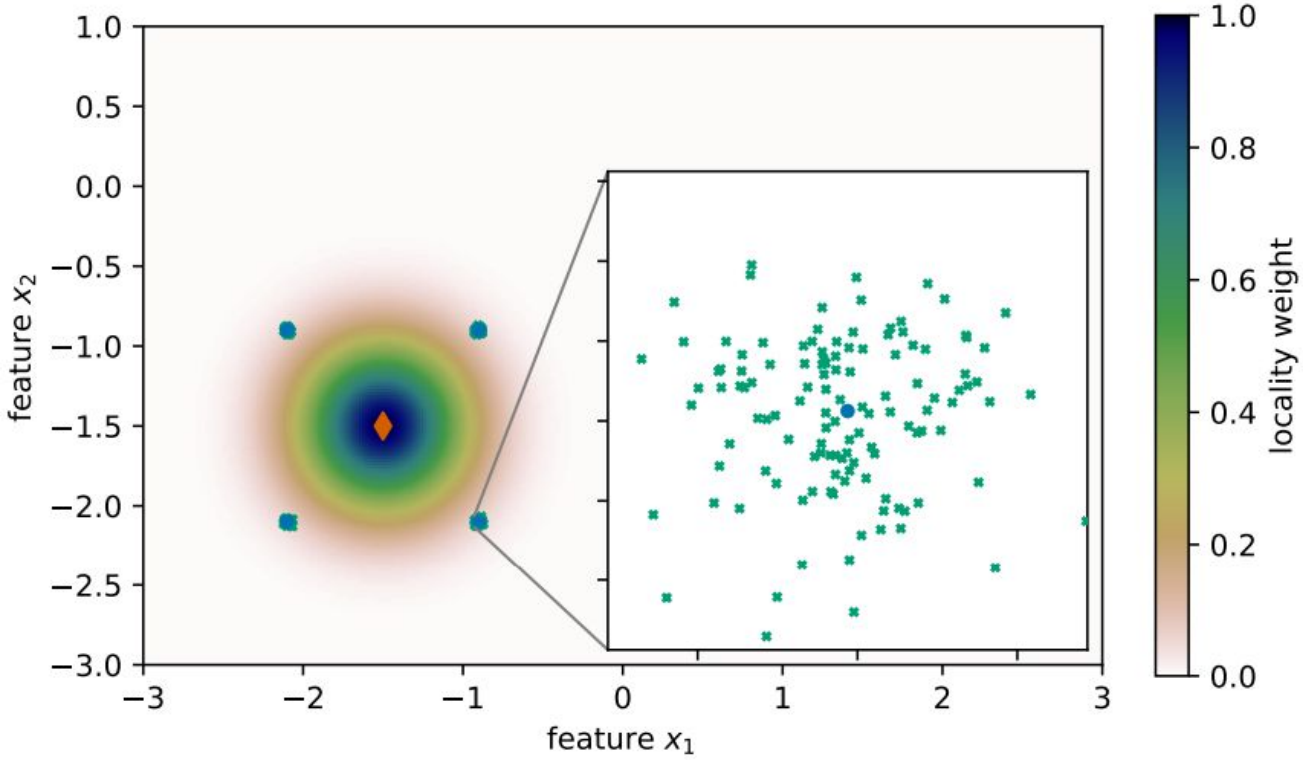


Figure 4: Enhanced sample by jittering around the support points

Green-LIME Example (contd.)

The reference point is shown as a red diamond. Nbd is defined using Euclidean distance and the RBF kernel.

The optimal design consists of support points (shown in blue) symmetrically surrounding the reference point (support pts with their locality weight ~ 1/18).

The total sample size is 501 points.

One sample is assigned to the reference point. And, four samples correspond to the optimal design support points.

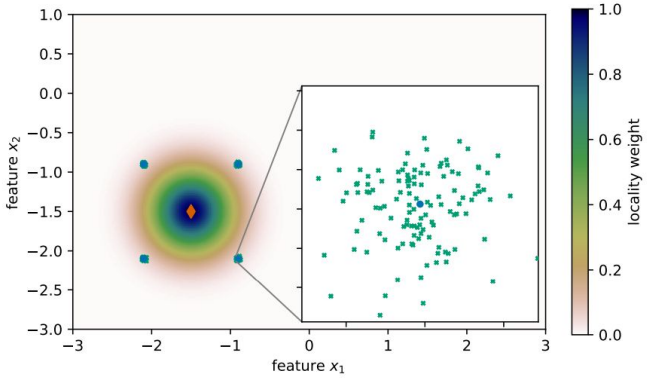


Figure 4: Enhanced sample by jittering around the support points

The remaining 496 samples are allocated to the support points proportional to their design weights.

These additional samples are generated by adding Gaussian noise with mean 0 and variance 0.01^2 from the support points (jittering).

Conclusion

- Optimal design of experiments improves LIME by replacing random sampling with structured, informative sampling.
- The proposed method reduces the number of model evaluations while maintaining comparable explanation quality.
- This leads to improved computational efficiency and lower energy consumption.
- The approach is currently for tabular data and regression setting.
- Optimal design provides a principled framework for efficient data collection in explainable AI

Future Directions

- Extend optimal design methods to higher-dimensional feature spaces using more efficient design constructions.
- Incorporate feature selection methods, such as penalized regression (e.g., Lasso), into the optimal design framework.
- Improve robustness by sampling from realistic data distributions rather than artificial perturbations.
- Develop joint optimal designs that generate explanations efficiently for multiple reference points.